Object-Oriented Programming (OOP) Fundamentals in Java

1. Why Do We Need Object-Oriented Programming?

1.1 Real-world entities have state and behavior

Consider a car. It has properties that describe its current condition, and it can perform actions.

State (data/properties)	Behavior (actions/methods)
Brand: Toyota	Start
Color: Red	Accelerate
Speed: 60 km/h	Apply brakes
Fuel: 30 litres	Stop

A car's speed belongs to that particular car. Similarly, accelerating is an action performed by that car. This gives us a useful way to model entities in software: **keep the data representing an entity and the operations performed on that data together**.

1.2 Representing a car using a procedural approach

In a simple procedural-style program, we could store the data in separate variables and pass it to functions.

```
String brand = "Toyota";
String color = "Red";
int speed = 0;

static int accelerate(int speed, int increase) {
    return speed + increase;
}

static int applyBrake(int speed, int decrease) {
    return Math.max(0, speed - decrease);
}
```

A complete example would look like this:

```
public class Main {
    public static void main(String[] args) {
        String brand = "Toyota";
        String color = "Red";
        int speed = 0;

        speed = accelerate(speed, 20);
        speed = accelerate(speed, 30);
        speed = applyBrake(speed, 10);

        System.out.println("Speed: " + speed); // 40
    }

    static int accelerate(int speed, int increase) {
        return speed + increase;
    }

    static int applyBrake(int speed, int decrease) {
        return Math.max(0, speed - decrease);
    }
}
```

This works fine for a small example. The problem becomes clearer as we add more cars and more properties.

1.3 What changes when there are multiple cars?

```
String brand1 = "Toyota";
int speed1 = 0;

String brand2 = "BMW";
int speed2 = 0;

String brand3 = "Honda";
int speed3 = 0;

speed1 = accelerate(speed1, 20);
speed2 = accelerate(speed2, 40);
speed3 = accelerate(speed3, 30);
```

Every new car requires another set of variables. It is also easy to mix up which values belong together:

```
speed1 = accelerate(speed2, 20); // Wrong car's speed used
```

The compiler cannot identify this particular mistake because `speed1` and `speed2` are both integers. With more properties—fuel, gear, engine state and so on—managing these relationships becomes harder.

This does **not** mean procedural programming cannot model real-world entities. Structures, records, modules and other techniques can organize procedural programs. The example simply shows the difficulty caused by keeping closely related data and operations separate.

1.4 The object-oriented approach

In OOP, a car can be represented as a single object that contains its own state and behavior.

Car object

- **State:** `brand`, `color`, `speed`
- **Behavior:** `accelerate()`, `applyBrake()`

Object-Oriented Programming (OOP) is an approach to software design in which objects combine data (state) with operations (behavior). It helps us work with related data and functionality as a single unit.

---

2. Classes and Objects

2.1 Class vs. object

A factory can use one design to manufacture multiple cars. The manufactured cars follow the same general design, but each car is a separate entity with its own properties.

Java follows a similar model:

- A **class** defines the fields and methods that objects of that type will have.
- An **object** is an instance created from that class.

```
class Car {
    String brand;
    String color;
    int speed;

    void accelerate(int increase) {
        speed = speed + increase;
    }

    void applyBrake(int decrease) {
        speed = Math.max(0, speed - decrease);
    }
}
```

Here, `brand`, `color` and `speed` are instance fields. `accelerate()` and `applyBrake()` are instance methods.

Unlike the procedural version, `accelerate()` does not require a separate `speed` argument. The method works with the `speed` field of the object on which it is called.

2.2 Creating different objects from the same class

```
Car car1 = new Car();
Car car2 = new Car();

car1.brand = "Toyota";
car1.color = "Red";
car1.speed = 0;

car2.brand = "BMW";
car2.color = "Blue";
car2.speed = 0;
```

In this example:

- `Car` is the class (the type).
- `new Car()` creates a new object.
- `car1` and `car2` are reference variables pointing to different objects.
- Each object has its own values for instance fields.

```
car1.accelerate(20);
car2.accelerate(50);

System.out.println(car1.speed); // 20
System.out.println(car2.speed); // 50
```

When we execute `car1.accelerate(20)`, Java runs the method with `car1` as the current object. The method updates `car1.speed`, not `car2.speed`.

Key distinction: The class describes the structure and behavior; each object has its own instance state.

---

3. Encapsulation

3.1 Why should an object control its data?

A driver controls a real car using operations such as the accelerator and brakes. They do not directly assign an arbitrary value to the car's speed.

However, consider this Java class:

```
class Car {
    String brand;
    int speed;

    void accelerate(int increase) {
        speed += increase;
    }
}
```

Because `speed` is directly accessible from code in the same package, someone could write:

```
Car car = new Car();
car.speed = -100;
car.speed = 100000;
```

For this model, speed is a non-negative magnitude, so `-100` is invalid. An unreasonably large speed could also violate the rules of the application. The

object has no control over these assignments.

3.2 What is encapsulation?

Encapsulation means keeping an object's related data and behavior together while controlling how its internal state can be accessed or changed.

In practice, outside code should use meaningful operations rather than changing internal fields indiscriminately.

Java provides access modifiers, including `private`, `public` and `protected`, to control accessibility.

```
class Car {
    private String brand;
    private int speed;

    public void accelerate(int increase) {
        speed += increase;
    }

    public void applyBrake(int decrease) {
        speed = Math.max(0, speed - decrease);
    }
}
```

External code cannot directly write `car.speed` when the field is private. It must use the operations made available by the class.

3.3 Private fields are not enough: methods need validation

Making a field private does not automatically make the object safe. This method still accepts an invalid input:

```
public void accelerate(int increase) {
    speed += increase;
}

// car.accelerate(-100); // Still problematic
```

The class should enforce its rules inside its methods. The following is a self-contained version of the car example with an **illustrative** maximum speed:

```
class Car {
    private static final int MAX_SPEED = 200;
    private int speed;

    public void accelerate(int increase) {
        if (increase <= 0) {
            throw new IllegalArgumentException(
                "Increase must be positive"
            );
        }

        if (increase > MAX_SPEED - speed) {
            throw new IllegalArgumentException(
                "Maximum speed exceeded"
            );
        }

        speed += increase;
    }

    public void applyBrake(int decrease) {
        if (decrease <= 0) {
            throw new IllegalArgumentException(
                "Decrease must be positive"
            );
        }
        speed = Math.max(0, speed - decrease);
    }

    public int getSpeed() {
        return speed;
    }
}
```

Now the object maintains the rules for its own state. This is the main benefit of encapsulation: **controlled access and protection of an object's valid state**.

---

4. Abstraction

4.1 Using something without knowing all its internals

To drive a car, we use controls such as the accelerator and ignition. We do not manually coordinate the fuel pump, engine control system and drivetrain every time we start or accelerate.

A complicated system becomes easier to use when it exposes simple operations and keeps implementation details behind those operations.

Abstraction means exposing the relevant capabilities of a component without requiring its user to understand every detail of how those capabilities are implemented.

4.2 Abstraction using an ordinary Java class

```
class Car {
    public void start() {
        activateFuelPump();
        activateIgnition();
        rotateEngine();
        System.out.println("Car started");
    }

    private void activateFuelPump() {
        System.out.println("Fuel pump activated");
    }

    private void activateIgnition() {
        System.out.println("Ignition activated");
    }

    private void rotateEngine() {
        System.out.println("Engine rotating");
    }
}
```

The caller only needs:

```
Car car = new Car();
car.start();
```

`start()` is the public operation. The helper methods are implementation details kept inside the class.

Important: Abstraction is a design idea. We can apply it with ordinary classes; interfaces and abstract classes are additional Java mechanisms for expressing it.

4.3 Interfaces: defining a common contract

Cars, bikes and buses may start and move differently, but they can share the same set of capabilities.

```
interface Drivable {
    void start();
    void accelerate();
    void brake();
}
```

The `Drivable` interface describes **what** an implementing type must be able to do, without defining **how** every vehicle performs those operations.

```
class Car implements Drivable {
    @Override
    public void start() {
        System.out.println("Car started using ignition");
    }

    @Override
    public void accelerate() {
        System.out.println("Car is accelerating");
    }

    @Override
    public void brake() {
        System.out.println("Car is braking");
    }
}
```

A `Bike` or `Bus` can implement the same interface with its own behavior. The interface provides a consistent contract across different implementations.

4.4 Abstract classes: sharing some implementation

Suppose petrol cars and electric cars share some information, such as a brand, but use different starting mechanisms.

An abstract class can provide the common fields and methods while requiring subclasses to implement specific behavior.

```
abstract class Car {
    private String brand;
```

```
    public Car(String brand) {
        this.brand = brand;
    }

    public void showBrand() {
        System.out.println(brand);
    }

    public abstract void start();
}

class ElectricCar extends Car {
    public ElectricCar(String brand) {
        super(brand);
    }

    @Override
    public void start() {
        System.out.println("Electric car starts");
    }
}
```

The abstract `Car` class provides shared functionality (`showBrand()`) but leaves the starting behavior for each concrete subclass. We cannot directly instantiate the abstract class.

In this example, the `Car` abstraction is separate from the earlier `Car` examples; each code block illustrates a different concept.

---

5. Inheritance

5.1 The IS-A relationship

A car, bike and bus are all vehicles. They may share common properties—such as a brand and speed—and operations such as moving and stopping.

That suggests a type hierarchy:

```
          Vehicle
         /   |   \
       Car  Bike  Bus
```

In Java, inheritance allows one class to extend another:

```
class Car extends Vehicle {
    // Car-specific fields and methods
}
```

This establishes a subtype relationship: **a `Car` IS-A `Vehicle`**.

A `Car` object includes the instance state associated with the `Vehicle` superclass and can use accessible inherited methods. Private fields in `Vehicle` are not directly accessible from methods in `Car`.

5.2 IS-A vs. HAS-A

Inheritance is most natural when the subclass genuinely represents a more specific kind of the parent type.

Relationship	Meaning	Typical design
Car **IS-A** Vehicle	A car is a kind of vehicle	Inheritance may be suitable
Bike **IS-A** Vehicle	A bike is a kind of vehicle	Inheritance may be suitable
Dog **IS-A** Animal	A dog is a kind of animal	Inheritance may be suitable
Car **HAS-A** Engine	An engine is a component of a car	Composition is usually more suitable

A car is **not** an engine. This distinction matters when choosing relationships in Low-Level Design. The IS-A test is a useful starting point, but it is not the only design consideration.

5.3 Types of inheritance in Java

Single inheritance — One class extends one parent class.

```
Vehicle -> Car
```

Multilevel inheritance — A subclass extends another subclass.

```
Vehicle -> Car -> ElectricCar
```

Hierarchical inheritance — Multiple classes extend the same parent class.

```
          Vehicle
         /   |   \
       Car  Bike  Bus
```

Multiple inheritance — Java does not allow a class to extend multiple classes. A class can, however, implement multiple interfaces.

---

6. Polymorphism

6.1 One operation, different behavior

A petrol car, electric car and motorcycle can all be started. The operation is called `start()`, but what happens internally depends on the vehicle.

Polymorphism allows a common method name or contract to work with different forms or types of objects, including type-specific behavior.

In Java, two commonly taught forms are:

1. **Compile-time polymorphism:** method overloading.
2. **Runtime polymorphism:** method overriding and dynamic dispatch.

6.2 Compile-time polymorphism: method overloading

Overloading means defining multiple methods with the same name but different parameter lists.

```
class Car {
    void accelerate(int increase) {
        System.out.println("Increase speed by " + increase);
    }

    void accelerate(int increase, int duration) {
        System.out.println(
            "Increase speed by " + increase +
            " for " + duration + " seconds"
        );
    }
}
```

Methods can be overloaded by changing:

- The number of parameters.
- The parameter types.
- The ordering of parameter types (where this produces a distinct signature).

Changing **only** the return type is not enough to overload a method.

6.3 Runtime polymorphism: method overriding

A subclass can supply its own implementation of an inherited instance method.

```
class Vehicle {
    public void start() {
        System.out.println("Vehicle started");
    }
}

class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("Car engine started");
    }
}

class Bike extends Vehicle {
    @Override
    public void start() {
        System.out.println("Bike engine started");
    }
}
```

Here, both `Car` and `Bike` override `Vehicle.start()`.

6.4 Parent reference, child object

```
Vehicle vehicle = new Car();
vehicle.start();
```

Output:

```
Car engine started
```

Although the reference variable has the declared type `Vehicle`, the actual object is a `Car`. For an overridden instance method, Java selects the implementation according to the actual object at runtime.

6.5 Why runtime polymorphism helps in Low-Level Design

Without a shared abstraction, an application might have separate methods such as `startCar()` and `startBike()`, or repeated conditions for each vehicle type.

With polymorphism:

```
void startVehicle(Vehicle vehicle) {
    vehicle.start();
}

startVehicle(new Car());
startVehicle(new Bike());
```

The same `startVehicle()` method can work with multiple subclasses. Each vehicle provides its own implementation of `start()`.

This is the connection between inheritance and polymorphism: **a common parent type lets code work with different child objects through the same operation**.

---

7. Putting the Four Pillars Together

The following compact example brings together the ideas covered above. It is a combined illustration based on the vehicle examples.

```
abstract class Vehicle {
    private final String brand;
    private int speed;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public String getBrand() {
        return brand;
    }

    public int getSpeed() {
        return speed;
```

```
    }

    public void accelerate(int increase) {
        if (increase <= 0) {
            throw new IllegalArgumentException("Invalid increase");
        }
        speed += increase;
    }

    public abstract void start();
}

class PetrolCar extends Vehicle {
    public PetrolCar(String brand) {
        super(brand);
    }

    @Override
    public void start() {
        System.out.println("Petrol engine started");
    }
}

class ElectricCar extends Vehicle {
    public ElectricCar(String brand) {
        super(brand);
    }

    @Override
    public void start() {
        System.out.println("Electric motor ready");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle car1 = new PetrolCar("Toyota");
        Vehicle car2 = new ElectricCar("Tesla");

        car1.start(); // Petrol engine started
        car2.start(); // Electric motor ready

        car1.accelerate(20);
        System.out.println(car1.getSpeed()); // 20
    }
}
```

Concept	Where it appears
Encapsulation	`brand` and `speed` are private; `accelerate()` controls how speed changes.
Abstraction	`Vehicle` exposes `start()` without specifying one universal starting mechanism.
Inheritance	`PetrolCar` and `ElectricCar` extend `Vehicle`.
Polymorphism	`Vehicle` references call different overridden `start()` methods depending on the actual object.

8. Quick Revision

Concept	Main idea	Java example
Class	Defines the fields and methods of a type	`class Car`
Object	An instance of a class, with its own state	`new Car()`
Encapsulation	Controls access to and modification of state	`private speed`, validated `accelerate()`
Abstraction	Exposes useful operations and hides internal complexity	`start()`, `interface Drivable`
Inheritance	Defines a subtype / IS-A relationship	`Car extends Vehicle`
Polymorphism	Enables common operations with different implementations	`Vehicle v = new Car(); v.start();`

Points to remember

- An object combines its state and behavior.
- A class defines a type; objects are instances of that type.
- Encapsulation is not just making fields private. The class should also enforce valid state changes.
- Abstraction is about the interface presented to users of a component, not only the `abstract` keyword.
- Inheritance models an IS-A relationship. HAS-A relationships usually call for composition instead.

- Overloading uses different method signatures; overriding lets subclasses replace inherited behavior.
- Runtime polymorphism lets code depend on a common type while the actual object determines the overridden method that runs.